

SOCIAL MEDIA AND WEB ANALYTICS

Prof Dr Matthias Bogaert

LECTURE 3: NATURAL LANGUAGE PROCESSING

CONTENT

- Lecture 3: Natural language processing
 - Introduction to NLP
 - Word embeddings
 - Concept
 - Word2vec
 - Glove
 - Visualization with t-SNE
 - Topic modeling

INTRODUCTION TO NLP

NATURAL LANGUAGE PROCESSING (NLP)

- Vast field of research
- **Definition:** the field of research that *analyzes* and *represents natural language* using computational techniques from *computer science*, *artificial intelligence*, *statistics*, and *linguistics* for the purpose of achieving *human-like language processing*
- Since 1990s: increase of the use of machine learning algorithms to automatically learn from linguistic data (text, speech)
- Problems in text mining for which NLP can help:
 - Contextual information (“This camera sucks.” vs. “This vacuum cleaner really sucks”)
 - Negations
 - Humor
 - Spelling
 - Multiple languages
 - etc.

MOST INTERESTING NLP TECHNIQUES

- Part-of-speech tagging (PoS-tagging)
 - Previous lecture
- Named entity recognition
 - Given a stream of text, determine which items in the text map to proper names, such as people or places
- **Word embeddings**
 - Mapping technique to map words/phrases/sentences/documents to vectors of real numbers
- **Topic modeling**
 - Discovery of hidden semantic structures in a text body
- **Sentiment analysis (next session)**
 - Derive the sentiment from texts

NAMED ENTITY RECOGNITION (NER)

- Subtask of information extraction
 - The extraction of structured information from unstructured data using unsupervised, semi-supervised and supervised learning techniques
 - NER = extraction of the named entities mentioned in a text
- **Goal:** *identify* named entities in a text and *correctly classify* them into the named entity types, given a *predefined set of entities* (persons, locations, organization, etc.)
- There is no common ground on what a ‘named entity’ should be
 - Often dependent upon the purpose and the domain of the application (e.g., healthcare)

NAMED ENTITY RECOGNITION (NER)

contentSkip to site indexPoliticsSubscribeLog InSubscribeLog InToday's PaperAdvertisementSupported **ORG** byF.B.I. Agent **Peter Strzok PERSON**,
Who Criticized Trump **PERSON** in Texts, Is FiredImagePeter Strzok, a top **F.B.I. GPE** counterintelligence agent who was taken off the special counsel
investigation after his disparaging texts about President **Trump PERSON** were uncovered, was fired. CreditT.J. Kirkpatrick **PERSON** for **The New York
TimesBy Adam Goldman ORG** and **Michael S. SchmidtAug PERSON**. **13 CARDINAL**, **2018WASHINGTON CARDINAL** — **Peter Strzok
PERSON**, the **F.B.I. GPE** senior counterintelligence agent who disparaged President **Trump PERSON** in inflammatory text messages and helped
oversee the **Hillary Clinton PERSON** email and **Russia GPE** investigations, has been fired for violating bureau policies, Mr. **Strzok PERSON**'s lawyer
said **Monday DATE**.Mr. Trump and his allies seized on the texts — exchanged during the **2016 DATE** campaign with a former **F.B.I. GPE** lawyer,
Lisa Page — in PERSON assailing the **Russia GPE** investigation as an illegitimate “witch hunt.” Mr. **Strzok PERSON**, who rose over **20 years
DATE** at the **F.B.I. GPE** to become one of its most experienced counterintelligence agents, was a key figure in **the early months DATE** of the
inquiry.Along with writing the texts, Mr. **Strzok PERSON** was accused of sending a highly sensitive search warrant to his personal email account.The
F.B.I. GPE had been under immense political pressure by Mr. **Trump PERSON** to dismiss Mr. **Strzok PERSON**, who was removed **last summer
DATE** from the staff of the special counsel, **Robert S. Mueller III PERSON**. The president has repeatedly denounced Mr. **Strzok PERSON** in posts on

APPROACHES TO NER

— Rule-based approaches (old school)

- Relies on manual grammar rules and specialized dictionaries
- **Pros:** High precision for specific lists; No training data needed
- **Cons:** Highly domain-specific (requires custom lexicons); Low recall (sensitive to variations)

— Traditional machine learning (traditional)

- Uses algorithms like conditional random fields or SVMs with **hand-crafted features** (e.g., "is capitalized", "suffix", "First word", etc.).
- **Pros:** Generalizes better than rules; Features are often explainable
- **Cons:** Requires tedious manual feature engineering; Performance stagnates on complex text

— Deep learning (modern)

- Neural networks that learn context automatically via embeddings.
- Implementations in SpaCy:
 - **Standard (en_core_web_sm/md):** Uses a **CNN** architecture. Fast & CPU-optimized
 - **Transformer (en_core_web_trf):** Uses **RoBERTa**. Highest accuracy but GPU-heavy
- **Pros:** Best performance (high precision & recall); No manual feature engineering needed
- **Cons:** Requires labeled training data; "Black box" (harder to interpret)

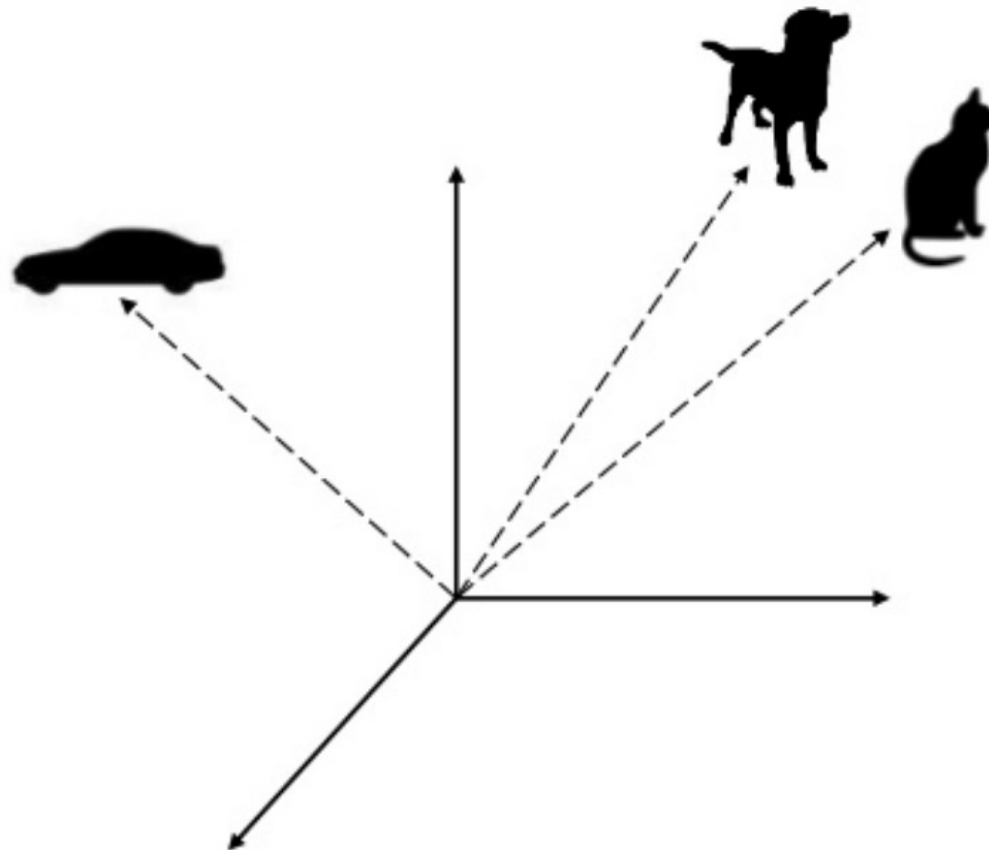
WORD EMBEDDINGS: CONCEPT

WORD EMBEDDINGS: CONCEPT

- What?
 - A numerical (vector) representation of a word
 - This numerical representation should be meaningful
 - We want to quantify *semantics* (or *context*)
 - E.g., the RGB representation of colors
- Simple embedding
 - One-hot encoded variables are most easy
 - Ghent [1 0 0 0 0]
- Better embedding
 - The elements represent proximity to other cities
 - Ghent [1 4 2 5]
 - The first element represents the difference to Aalst, second to Brussels, third to Bruges and fourth to Antwerp

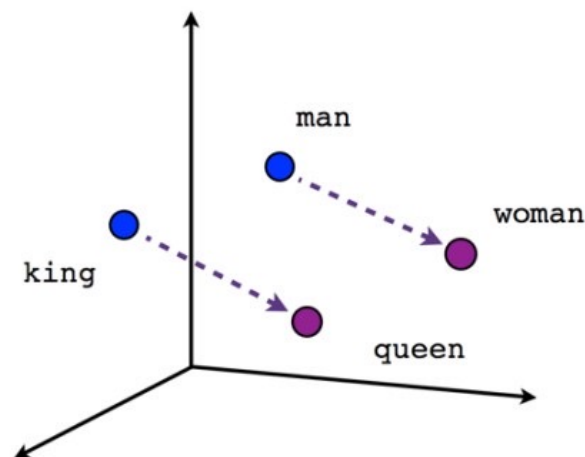
WORD EMBEDDINGS: CONCEPT

- Definition
 - A *dense* vector of real values, for which the dimension is typically much smaller than the vocabulary size and preserve the *semantic* relationship between words
 - E.g. by default: 128 dimensions
- Imagine embeddings as coordinates for times in the embedding space.

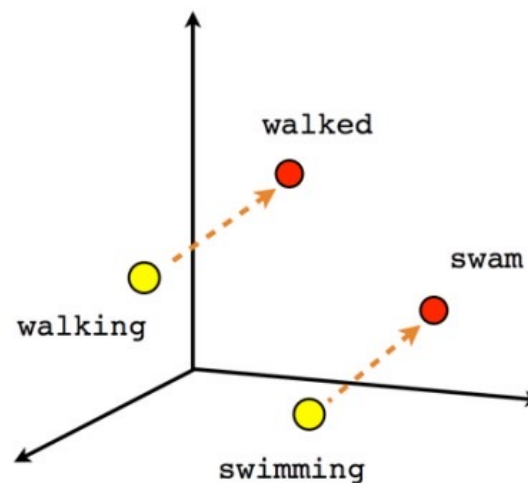


WORD EMBEDDINGS: CONCEPT

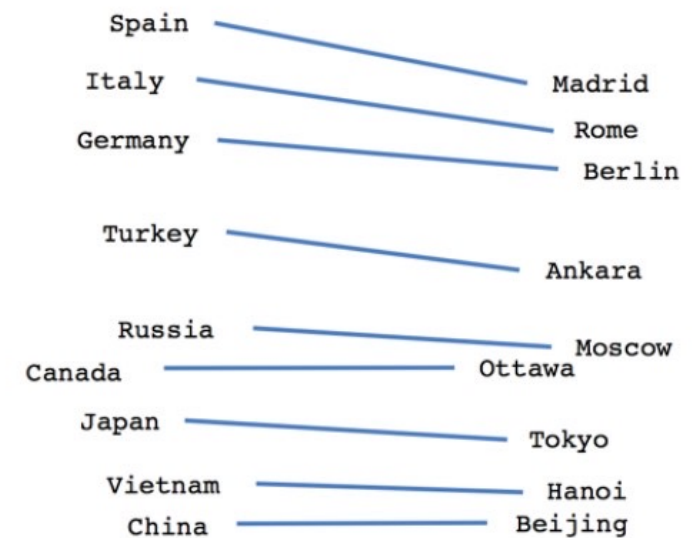
- Intuition
 - We want to keep the semantic relationship (or context) between words
 - Word context vectors are very good in preserving the analogies between words
- Example
 - *Man* is to *woman* as *king* is to ____?
 - *Good* is to *best* as *smart* is to ____?
 - *Belgium* is to *Brussels* as *France* is to ____?



Male-Female



Verb tense



Country-Capital

WORD EMBEDDINGS: CONCEPT

- Analogies are determined by the number of times that words are used in the same context
- For example
 - I like to pet my ...
 - *Dog*
 - *Cat*
 - *Laptop*

WORD EMBEDDINGS: TECHNIQUES

- Two main strategies to learn “lower-dimensional” dense vectors for representing words using the same intuition (i.e., word embeddings)
 - Neural network-based
 - *Word2Vec*
 - Matrix factorization-based
 - LSI
 - *GloVe*

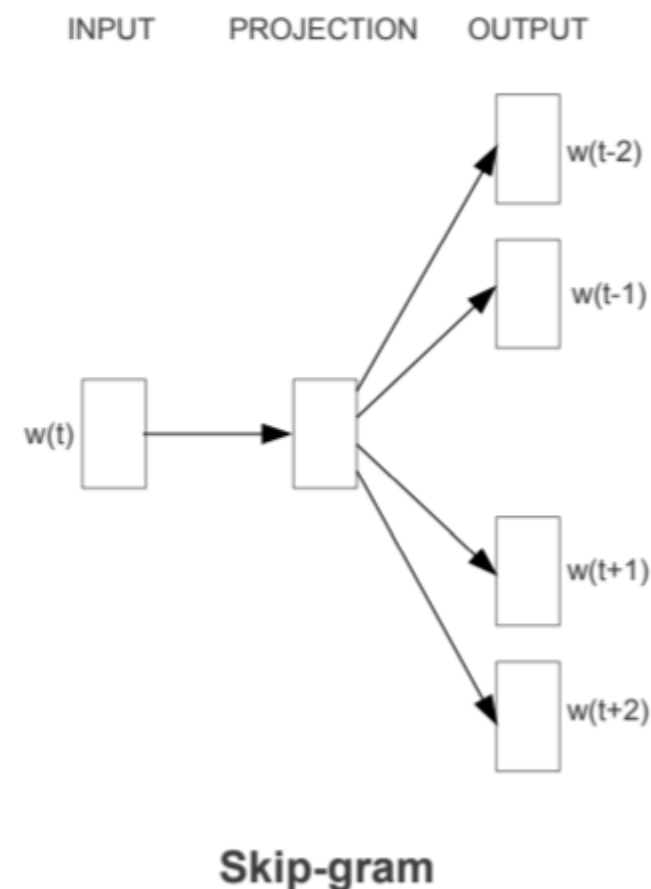
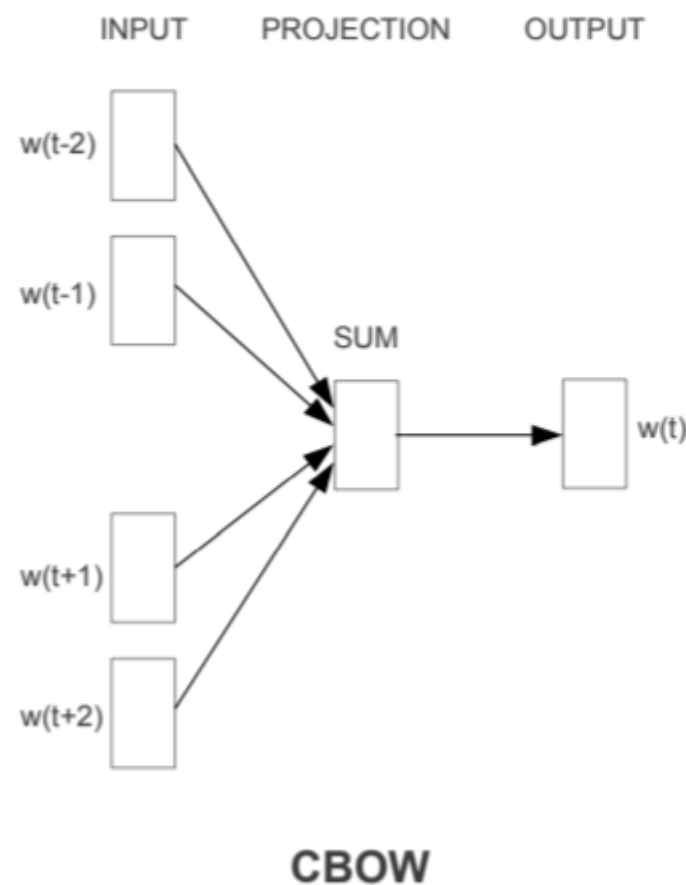
WORD EMBEDDINGS: WORD2VEC

WORD2VEC

- Designed by Google in 2013
- Neural network-based techniques that learn vector embeddings
- Steps:
 1. Context representation: create input and output one-hot encoded vectors from text that can be used as training data
 2. Select a model to train the one-hot vectors
 3. Define a loss function to optimize the model
 4. Evaluate the model by ensuring that similar words have similar meanings

WORD2VEC: CONTEXT REPRESENTATION

- **SkipGram**: trains each context against the word.
 - “Given this single word, what are the other words that are likely to appear near it at the same time?”
- **Continuous bag of words (CBOW)**: trains each word against its context.
 - “Given this set of context words, what missing word is likely to also appear at the same time?”



WORD2VEC: CONTEXT REPRESENTATION

— SkipGram

- Given a specific word (target or input word) what are the chances that a specific word is nearby
- Creation of training samples (i.e., input-output pairs) for our prediction model
- Window size: how many words behind and ahead of the input word do we go
 - Typical: 5 to 10
- Output probabilities: how likely is it to find each word close to the input word

WORD2VEC: CONTEXT REPRESENTATION

— Skipgram

- Window size = 2

Text	Training samples
The <u>dog</u> <u>barks</u> at the doorbell	(the, dog)
	(the, barks)
<u>The</u> dog <u>barks</u> <u>at</u> the doorbell	(dog, the)
	(dog, barks)
	(dog, at)
<u>The</u> <u>dog</u> barks <u>at</u> <u>the</u> doorbell	(barks, the)
	(barks, dog)
	(barks, at)
	(barks, the)

WORD2VEC: CONTEXT REPRESENTATION

— Skipgram

— What is the problem?

Text	Input	Output	Label
The <u>dog</u> <u>barks</u> at the doorbell	the	Dog	1
	the	Barks	1
<u>The</u> dog <u>barks</u> <u>at</u> the doorbell	dog	The	1
	dog	Barks	1
	dog	At	1
<u>The</u> <u>dog</u> barks <u>at</u> <u>the</u> doorbell	barks	The	1
	barks	Dog	1
	barks	At	1
	barks	the	1

WORD2VEC: CONTEXT REPRESENTATION

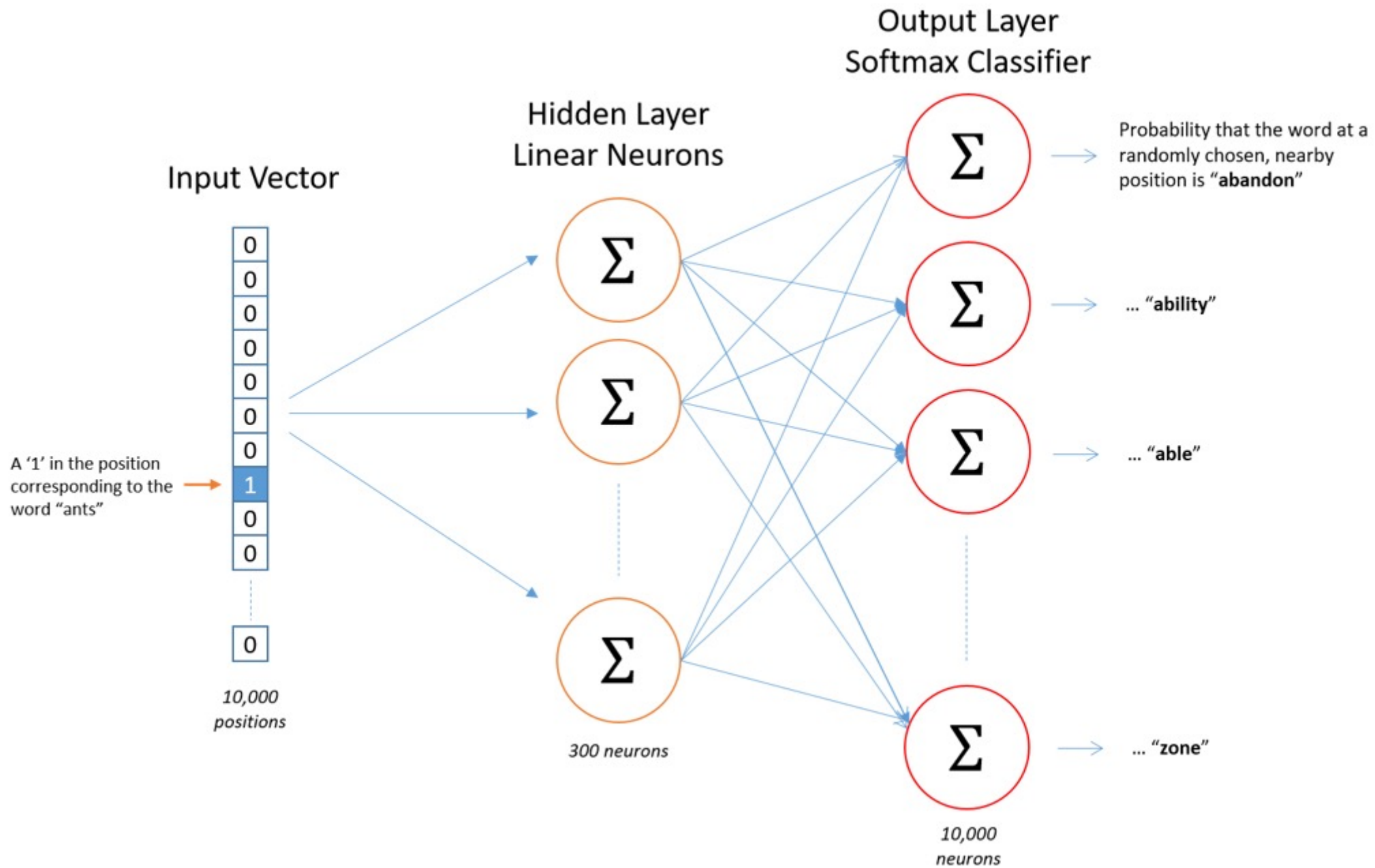
— Skipgram with negative sampling!

Text	Input	Output	Label
<i>The dog</i> barks <i>at the</i> doorbell	barks	The	1
	barks	taco	0
	barks	smile	0
	barks	Dog	1
	barks	phone	0
	barks	book	0
	barks	At	1
	barks	baby	0
	barks	grow	0
	barks	the	1
	barks	help	0
	barks	care	0

WORD2VEC: LEARNING MODEL

- Shallow neural network with one hidden layer and output layer
 - Hidden layer: no activation function
 - Output function: softmax
- Input layer
 - Every word in the vocabulary is represented as a one-hot vector
 - If you have 10,000 words then this would be a vector with size 10,000 which resolves to 1 for the corresponding word and 0 otherwise
- Output layer
 - Single vector with for every word the probability that a randomly selected nearby word is that vocabulary word

WORD2VEC: LEARNING MODEL



WORD2VEC: LEARNING MODEL

— Hidden layer

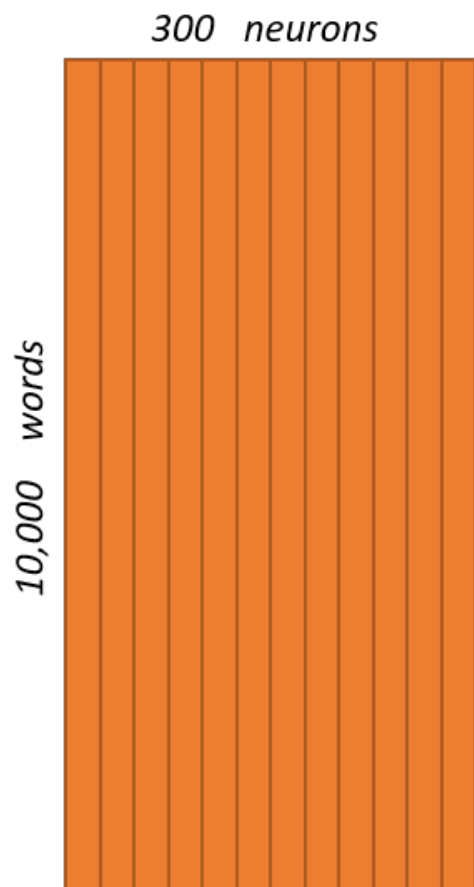
- The hidden layer can be seen as a weight matrix that will transform in the input vector to a proper output vector.
 - In the original paper by Google, 300 hidden layers were used to train the model on the News data set.
 - Number of hidden layers = hyper-parameter that should be tuned
 - In essence, the hidden layer can be seen as a word look-up matrix

WORD2VEC: LEARNING MODEL

Hidden Layer
Weight Matrix



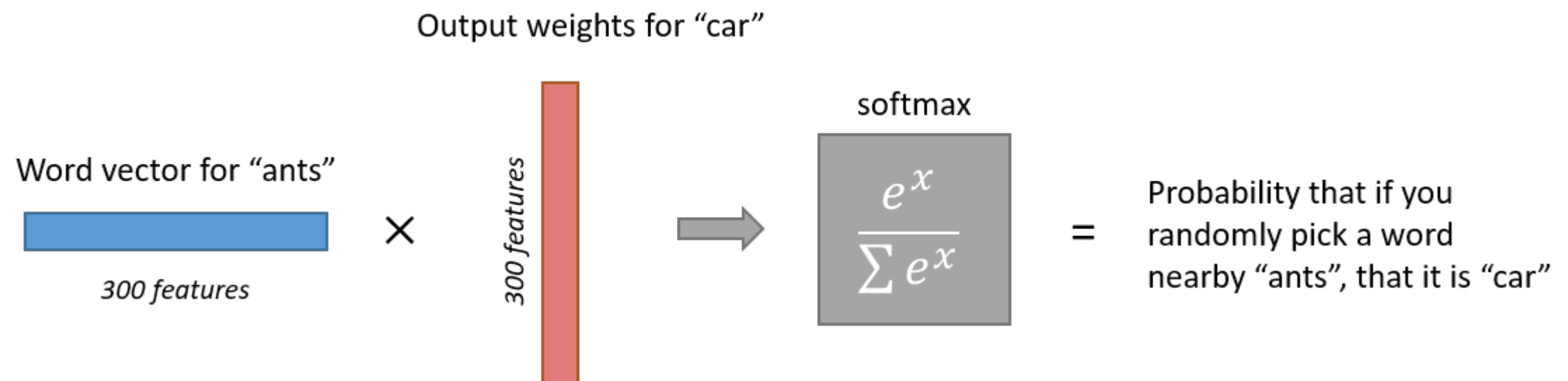
*Word Vector
Lookup Table!*



$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 \end{bmatrix} \times \begin{bmatrix} 17 & 24 & 1 \\ 23 & 5 & 7 \\ 4 & 6 & 13 \\ 10 & 12 & 19 \\ 11 & 18 & 25 \end{bmatrix} = \begin{bmatrix} 10 & 12 & 19 \end{bmatrix}$$

WORD2VEC: LEARNING MODEL

- Output layer
 - A softmax regression function
 - Comparable to the sigmoid/logistic function
 - Transforms the output values between 0 and 1 and sums up to 1

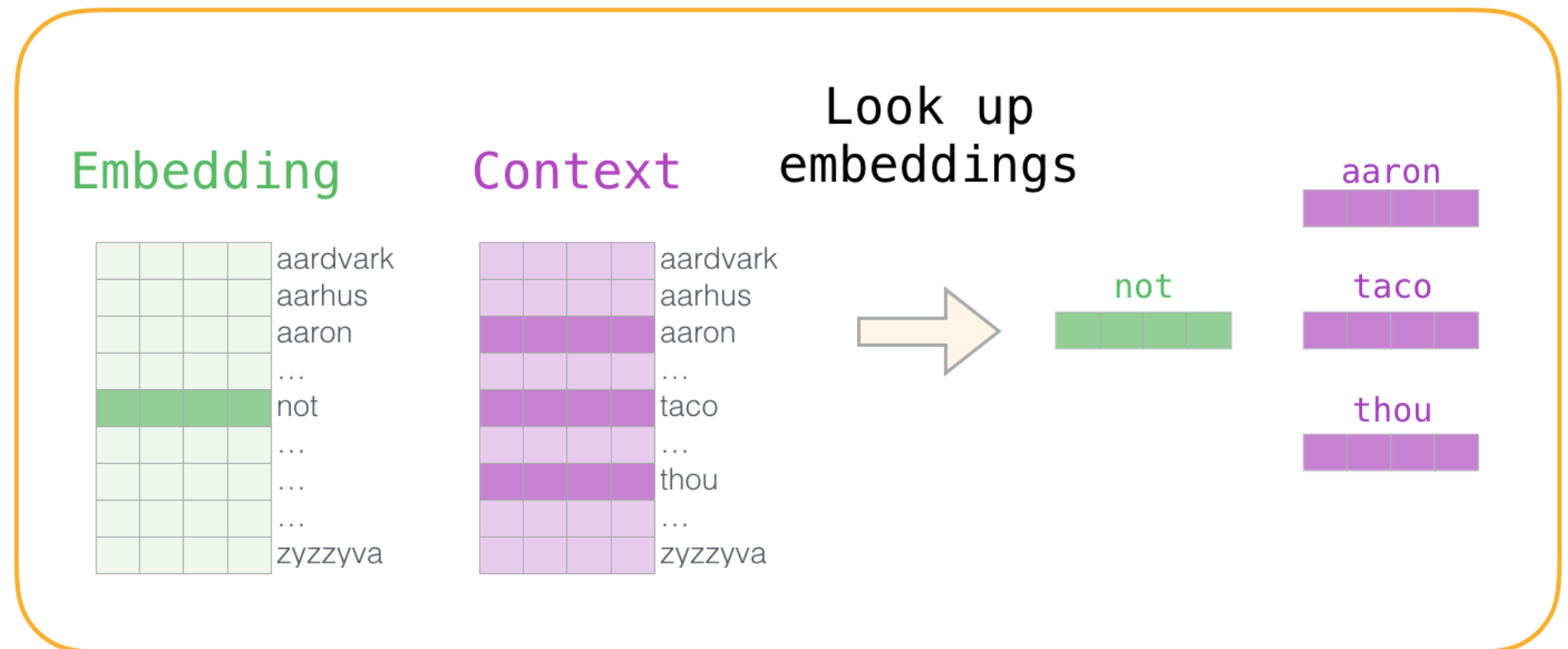
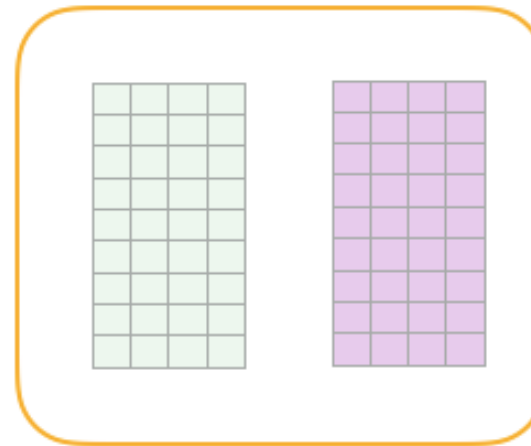


WORD2VEC: TRAINING PROCESS

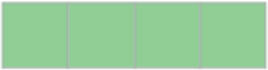
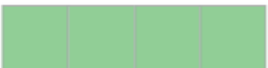
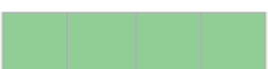
dataset

input word	output word	target
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	mango	0
not	finglonger	0
not	make	1
not	plumbus	0
...	...	...

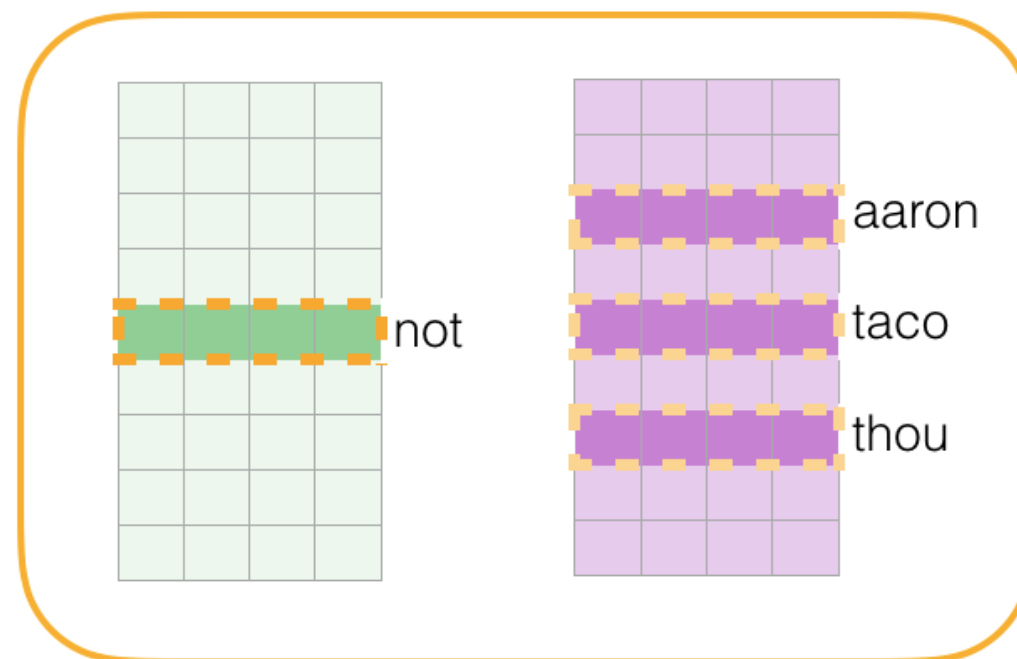
model



WORD2VEC: TRAINING PROCESS

input word	output word	target	input • output	sigmoid()	Error
not 	thou 	1	0.2	0.55	0.45
not 	aaron 	0	-1.11	0.25	-0.25
not 	taco 	0	0.74	0.68	-0.68

Only keep the
optimized green
embedding as your
pre-trained layer



**Update
Model
Parameters**

WORD EMBEDDINGS: GLOVE

- GLObal Vectors for word representation
- Designed by the Stanford NLP group as their answer to Word2Vec
- Does not rely on local statistics (i.e., local context of the words)
- Uses global statistics: it is trained on the non-zero entries of the global word *co-occurrence matrix*
 - How many times does word X co-occur with word Y
- Comparable to other matrix factorization methods, such as LSI

GLOVE

— Downside Word2Vec

- Only learns from the local information surrounding a word
- For example
 - *The student learns from the lecturer*
 - “The”
 - ?special context of *student* and *lecturer*
 - ?a simple stopword

— GloVe

- Traditional methods such as LSI do not capture word analogies that well from global statistics
- GloVe tries to combine the best of both worlds by deriving semantics from the co-occurrence matrix

GLOVE: CO-OCCURRENCE MATRIX

- Given a corpus of size N , the co-occurrence matrix X is a matrix of size $N \times N$
 - X_{ij} represents the co-occurrence of word i with word j
 - *The student learns from the lecturer*

	The	Student	Learns	From	Lecturer
The	0	1	0	1	1
Student	1	0	1	0	0
Learns	0	1	0	1	0
From	1	0	1	0	0
Lecturer	1	0	0	0	0

GLOVE: CO-OCCURRENCE MATRIX

- Similarity
 - Assume 3 words at a time and co-occurrence matrix with probabilities P
 - P_{ik} represents the propensity of seeing word i and k together ($= \frac{X_{ik}}{X_i}$)
 - Word k is called the probe word
 - $\frac{P_{ik}}{P_{jk}} \gg 1$ (or $\frac{P_{ik}}{P_{jk}} \ll 1$) the probe word is similar to i (j) but not to j (i)
 - $\frac{P_{ik}}{P_{jk}} \approx 1$ the probe word is (not) related to both words

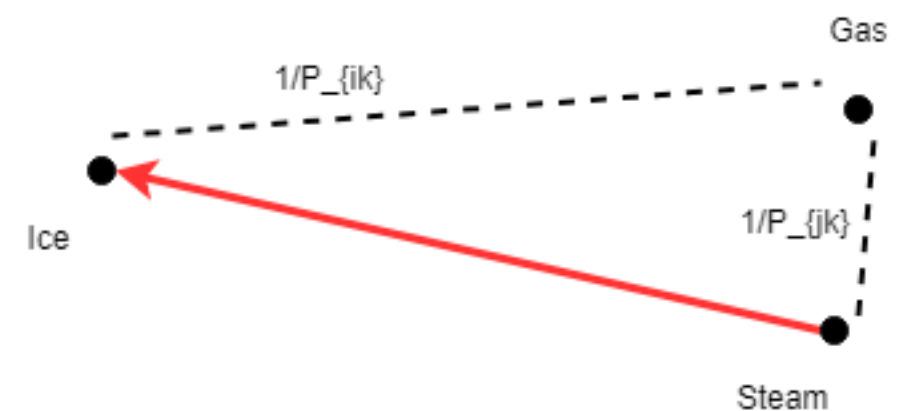
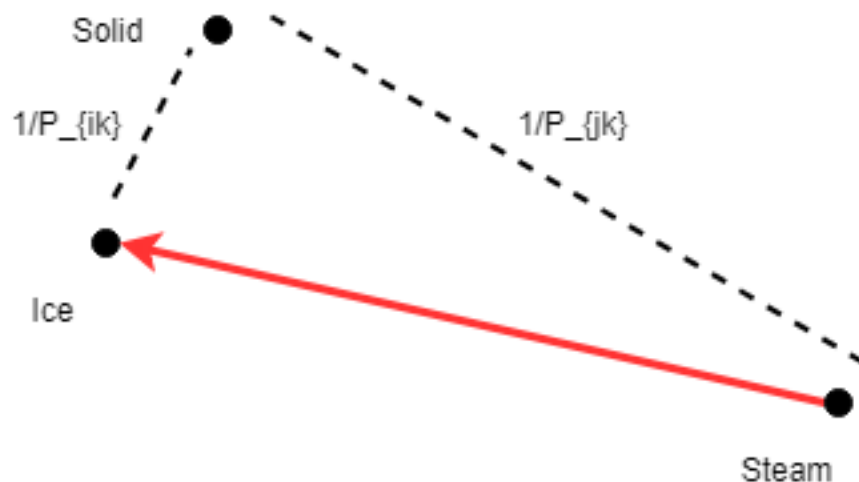
Probability and Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k ice)/P(k steam)$	8.9	8.5×10^{-2}	1.36	0.96

GLOVE: ALGORITHM

- How to go from these metrics to word embeddings?
 - There is no simple equation: $F(i, j, k) = P_{ik}/P_{jk}$
 - Word embeddings are vectors, whereas P_{ik}/P_{jk} is a scalar
 - The 3 entities (i, j, k) have to be transformed to 2 to compute a loss function
- Assume
 - w_i, w_j, u_k : two unique word embedding layers (randomly initialized)
 - X : the co-occurrence matrix
 - b_w and b_u : the biases of w and u

GLOVE: ALGORITHM

- Create a function F that takes the word vectors as input
 - $F(w_i, w_j, u_k) = P_{ik}/P_{jk}$
- Since word vectors are linear systems
 - $F(w_i - w_j, u_k) = P_{ik}/P_{jk}$
 - In words, the distance between two words i (j) and k is correlated with the reciprocal of P_{ik} (P_{jk})



GLOVE: ALGORITHM

- From vector to scalar
 - Take the dot product of the two word vectors
 - $F((w_i - w_j)^T \cdot u_k) = P_{ik} / P_{jk}$
 - Assume *homomorphism* for F between multiplicative groups and additive groups
 - $f(x \cdot y) = f(x) + f(y)$
 - $f(x - y) = f(x) / f(y)$
 - $F((w_i - w_j)^T \cdot u_k) = \frac{F(w_i^T u_k)}{F(w_j^T u_k)} = \frac{P_{ik}}{P_{jk}}$
 - $F(w_i^T u_k) = P_{ik}$

GLOVE: ALGORITHM

- Assume $F = \exp$
 - $w_i^T u_k = \log(P_{ik}) = \log(X_{ik}) - \log(X_i)$
 - $w_i^T u_k + \log(X_i) = \log(X_{ik})$
- Since $\log(X_i)$ is independent of k , this term can be absorbed in the bias term
 - $w_i^T u_k + b_w + b_u = \log(X_{ik})$
- Gives us a mean square **loss function**
 - $J = (w_i^T u_k + b_w + b_u - \log(X_{ik}))^2$

GLOVE: ALGORITHM

- Instead of changing $\log(X_{ik}) \rightarrow \log(1 + X_{ik})$, add a weighting function $f(x)$

- $$f(x) = \begin{cases} \left(\frac{x}{x_{max}}\right)^\alpha & \text{if } x < x_{max} \\ 1 & \text{otherwise,} \end{cases}$$

- $x_{max} = 100$ and $\alpha = \frac{3}{4}$

- The final **cost function** becomes

- $$J = \sum_{i,j=1}^N f(X_{ij}) (w_i^T u_j + b_w + b_u - \log(X_{ij}))^2$$

- This can be solved with least squares using the Adagrad optimizer
- GloVe boils down to a global log-bilinear regression model

COMPARING WORD EMBEDDINGS

- Numerical

- Cosine similarity between two vectors a and b

- $$\text{Cos}(\theta) = \frac{a \cdot b}{\|a\| \cdot \|b\|} = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2} \sqrt{\sum_{i=1}^n b_i^2}}$$

- Values: -1: opposite, 0: orthogonal, 1: equal

- Graphical

- **t-Distributed Stochastic Neighbor Embedding (t-SNE)**
 - Technique for dimensionality reduction that is particularly well suited for the visualization of high-dimensional datasets

WORD EMBEDDINGS: TSNE

VISUALIZATION WITH T-SNE

- t-SNE: t-distributed stochastic neighbor embedding (van der Maaten and Hinton, 2008)
- t-SNE is a dimensionality reduction technique
 - Comparable to SVD and PCA
 - However, t-SNE tries to preserve local similarities (i.e., small pairwise distances)
- t-SNE is a nonlinear dimensionality reduction technique based on manifold learning
 - Each data point lies on an embedded non-linear manifold within a higher dimensional space
 - A manifold is a topological space that locally resembles a Euclidean space near each data point
 - E.g., a surface is 2 dimensional manifold, locally resembling a Euclidean plane near each point
 - A higher dimensional space can thus be well “embedded” in a lower dimensional space
- Other manifold learning techniques: multi-dimensional scaling (MDS), isomap, auto-encoders, etc.

VISUALIZATION WITH T-SNE

— Two step procedure:

1. Construct a probability distribution representing a similarity measure over pairs of high-dimensional data points
2. Construct a similar probability distribution over the data points in the low-dimensional map

— Similarity is defined by using the Kullback-Leibler divergence (i.e., information gain or relative entropy)

— Assume the probability density distribution p and q , then the Kullback -Leibler divergence is computed as:

$$D_{KL}(p||q) = \sum_x p(x) \log\left(\frac{p(x)}{q(x)}\right)$$

— The goal is to minimize the divergence between two probability distributions

VISUALIZATION WITH T-SNE

1. Measure the similarities between data points in the original (high dimensional) space
 - The similarity of x_j to x_i is the conditional probability, $p_{j|i}$ that x_i would pick x_j as its neighbor if the neighbors were picked in proportion to their probability density assuming a Gaussian centered at x_i .
 - Next, measure the density of all other data points under a Gaussian distribution and normalize to make sure the probabilities sum to 1

$$- \quad p_{j|i} = \frac{e^{\frac{-\|x_i - x_j\|^2}{2\sigma_i^2}}}{\sum_{k \neq i} e^{\frac{-\|x_i - x_k\|^2}{2\sigma_i^2}}}$$

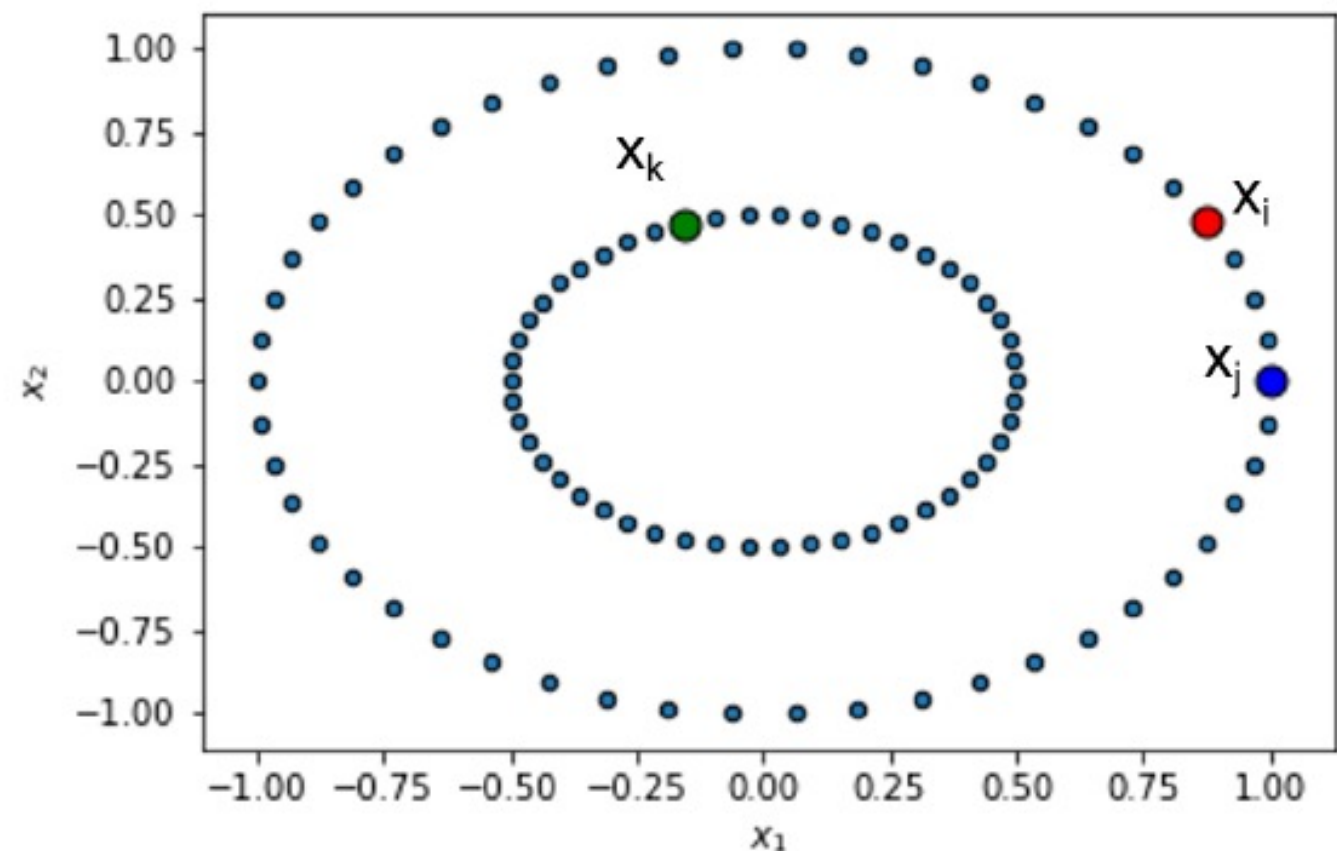
$$- \quad p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}$$

$$- \quad p_{i|i} = p_{ii} = 0$$

VISUALIZATION WITH T-SNE

- Consider 3 data points: x_i, x_j, x_k :
 - Compute the conditional and joint probabilities using the formulas:

$$- p_{j|i} = \frac{e^{\frac{-\|x_i - x_j\|^2}{2\sigma_i^2}}}{\sum_{k \neq i} e^{\frac{-\|x_i - x_k\|^2}{2\sigma_i^2}}}$$
$$- p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N} \text{ with } N = 2$$



VISUALIZATION WITH T-SNE

— Consider 3 data points: x_i, x_j, x_k :

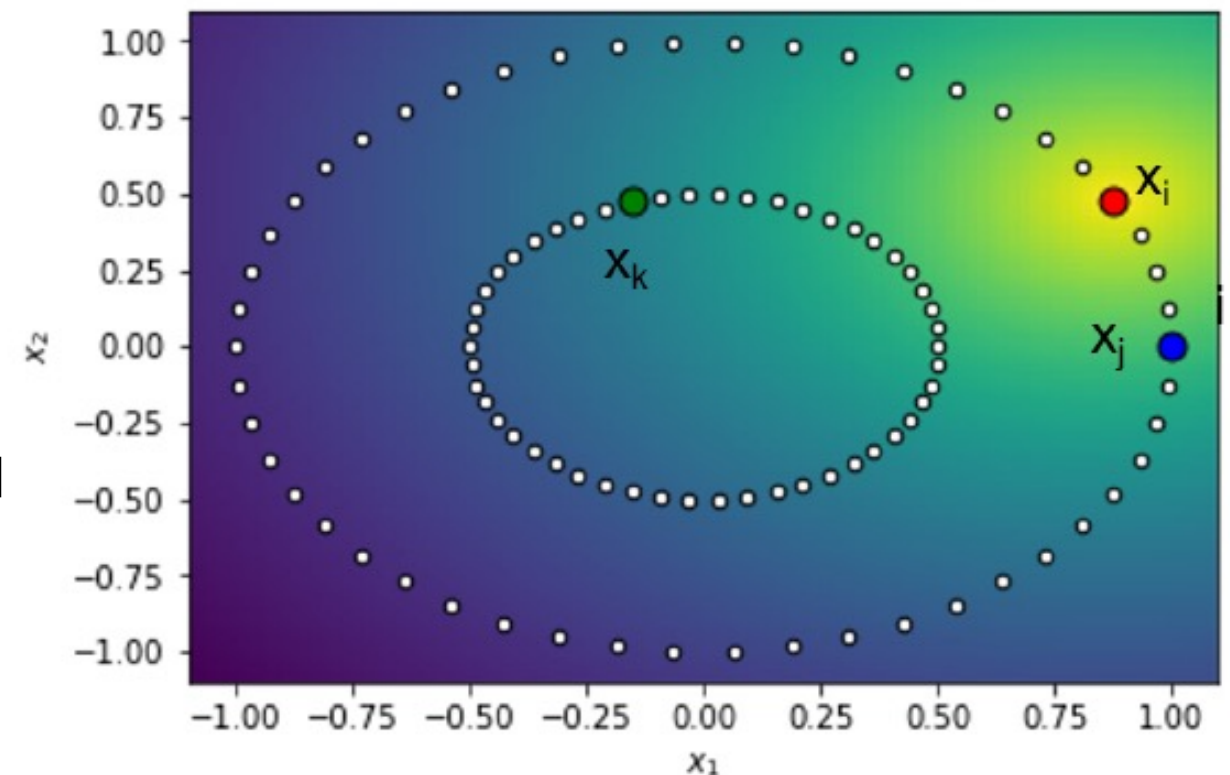
$$- p_{j|i} = \frac{e^{\frac{-||x_i - x_j||^2}{2\sigma_i^2}}}{\sum_{k \neq i} e^{\frac{-||x_i - x_k||^2}{2\sigma_i^2}}}$$
$$- p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N} \text{ with } N = 2$$

— **Nominator:** Gaussian probability centered at x_i

— Calculate conditional probabilities

$$- p_{j|i} = \frac{0.78}{z}$$

$$- p_{k|i} = \frac{0.60}{z}$$



VISUALIZATION WITH T-SNE

— Consider 3 data points: x_i, x_j, x_k :

$$- p_{j|i} = \frac{e^{-\frac{\|x_i - x_j\|^2}{2\sigma_i^2}}}{\sum_{k \neq i} e^{-\frac{\|x_i - x_k\|^2}{2\sigma_i^2}}}$$

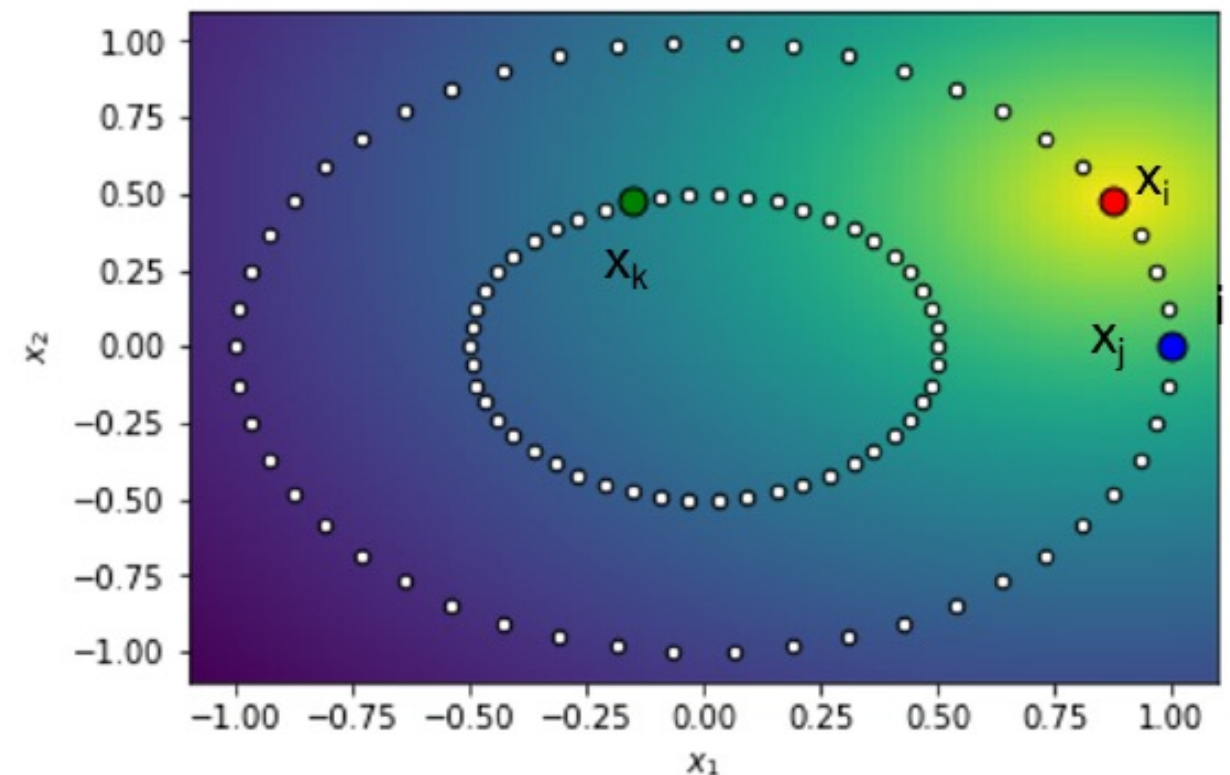
$$- p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N} \text{ with } N = 2$$

— **Denominator**: distance values against all points except for i

— Calculate conditional probabilities

$$- p_{j|i} = \frac{0.78}{55.62} = 0.014$$

$$- p_{k|i} = \frac{0.60}{55.62} = 0.011$$



VISUALIZATION WITH T-SNE

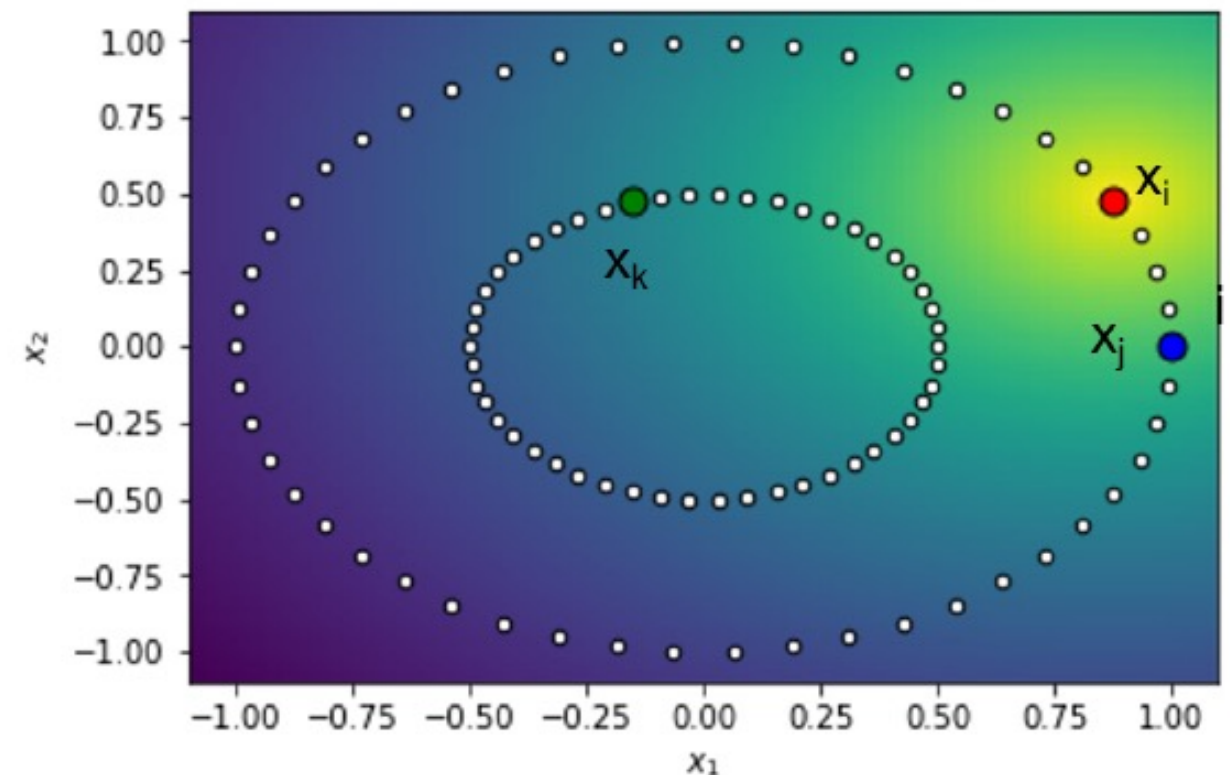
— Consider 3 data points: x_i, x_j, x_k :

$$p_{j|i} = \frac{e^{-\frac{\|x_i - x_j\|^2}{2\sigma_i^2}}}{\sum_{k \neq i} e^{-\frac{\|x_i - x_k\|^2}{2\sigma_i^2}}}$$
$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N} \text{ with } N = 2$$

— Calculate the joint probabilities

$$p_{ij} = 0.007$$

$$p_{ik} = 0.0055$$



VISUALIZATION WITH T-SNE

1. Recap: measure the similarities between data points in the original (high dimensional) space

$$- p_{j|i} = \frac{e^{\frac{-||x_i - x_j||^2}{2\sigma_i^2}}}{\sum_{k \neq i} e^{\frac{-||x_i - x_k||^2}{2\sigma_i^2}}}$$

- Based upon Euclidean distances
- σ_i (bandwidth of the Gaussian kernel) is data-point dependent
 - Based upon the **perplexity** which is a measure to estimate how well the distribution predicts a sample
 - Perplexity balances t-SNE between local and global aspects of your data and is comparable to the number of close nearest neighbors each data point has
 - Perplexity is one of the key tunable parameters in t-SNE
 - σ_i is set in such a way that the perplexity of the conditional distribution equals a predefined perplexity
 - σ_i depends on the density of the data: smaller values in denser parts of the data

VISUALIZATION WITH T-SNE

2. Measure the similarities between data points in the low dimensional space

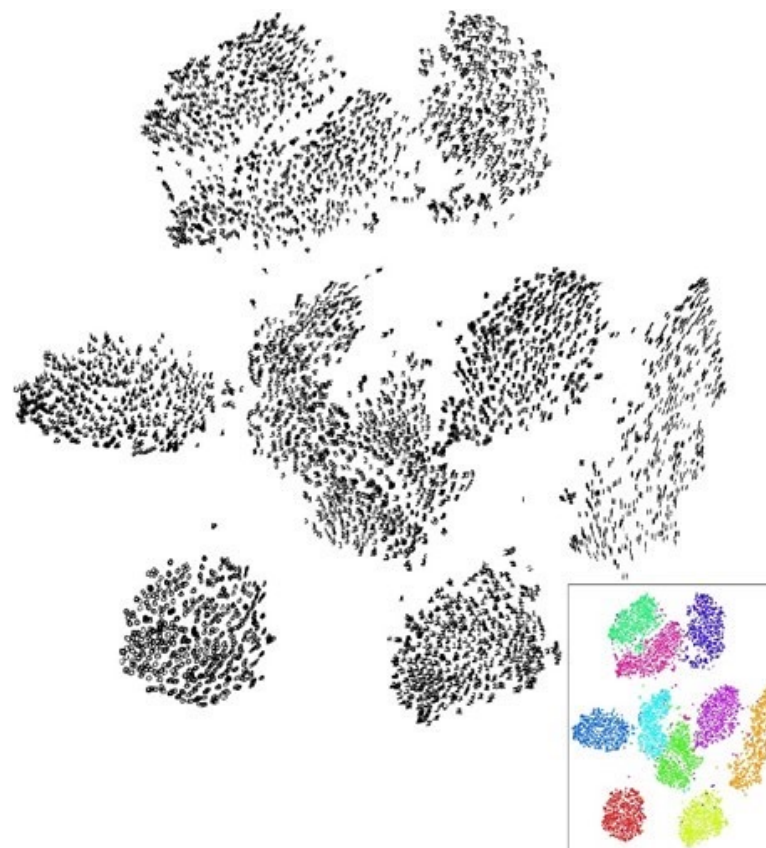
- Calculate q_{ij} between the mapped data points $y_i, y_j, \dots$
- Student's t-distribution is used to measure the similarities with the degrees of freedom equal to the dimensionality in the mapped space minus 1
- Student t-distribution has fatter tails than a Gaussian one
- Again: $q_{i|i} = q_{ii} = 0$
- No perplexity!

VISUALIZATION WITH T-SNE

- Next, measure the distance between p_{ij} and q_{ij}
 - q_{ij} depends on the locations of the data points in the mapped space
 - These locations are determined by minimizing the Kullback-Leibler divergence:
 - $D_{KL}(p||q) = \sum_{i \neq j} p_{ij} \log(\frac{p_{ij}}{q_{ij}})$
 - Optimized using gradient descent

VISUALIZATION WITH T-SNE

- t-SNE is especially useful when working with high dimensional data
 - Images and word documents (embeddings)
- Be aware: t-SNE is **not** a clustering technique!
 - A 2nd level clustering on the mapped space can be performed easily using K-means or any other clustering algorithm
 - It can also be seen as a form of feature engineering!



VISUALIZATION WITH T-SNE

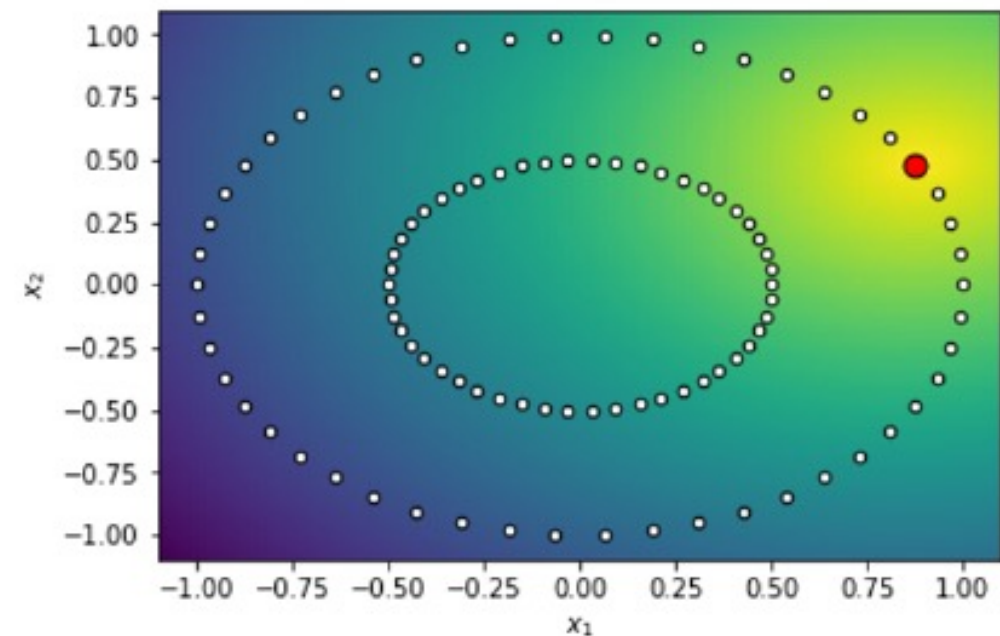
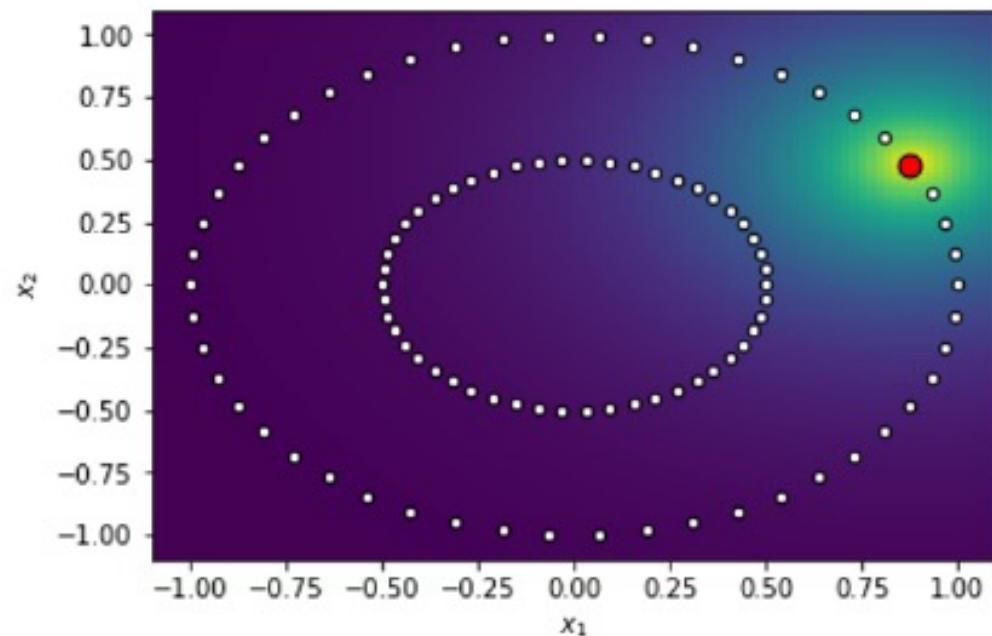
- Most implementations only allow for 2D or 3D dimensions
- t-SNE is a nonparametric mapping technique
 - No explicit mapping function from input to output space
 - Good for exploration but less for feature engineering in a predictive pipeline
 - Parametric t-SNE: learn a multivariate regression that predicts the map location from the input data
- You can also provide your own similarity matrix and run the KL minimization
 - Diagonal elements should be zero and the matrix elements should be normalized
 - Distance matrix can also be used when $\text{similarity} = 1 - \text{distance}$
 - Could be a workaround to avoid tuning the perplexity (however what is a good similarity metric?)

VISUALIZATION WITH T-SNE

- Since t-SNE uses gradient descent, general remarks regarding learning rates and initialization still hold
 - Sometimes PCA are used as initial starting points
 - However, defaults work well
- Most important hyperparameter: **perplexity**
 - A knob that sets the number of effective nearest neighbors (k in KNN)
 - Depends on the density of the data: the denser the data, the larger perplexity
 - Typical values: [5:50]

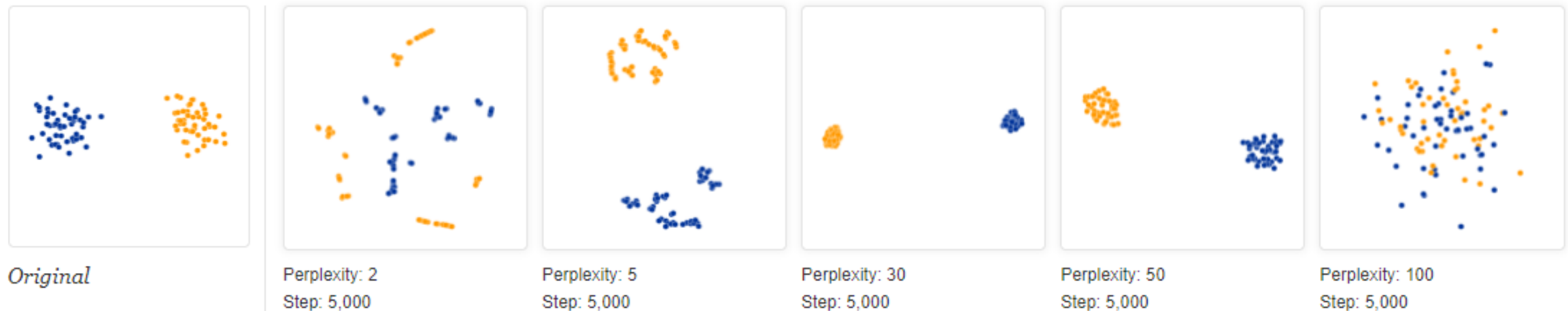
VISUALIZATION WITH T-SNE

- Impact of perplexity parameter
 - The effective nearest neighbors considered
 - Low vs high perplexity



VISUALIZATION WITH T-SNE

- Impact of perplexity parameter
 - The results are sensitive to the choice of the perplexity parameter
 - Perplexity should not be bigger than the number of data points



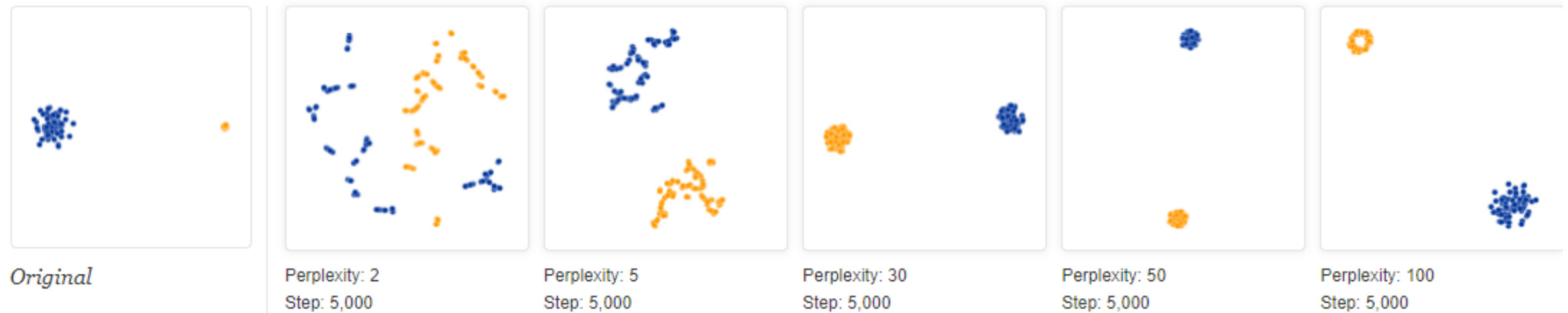
VISUALIZATION WITH T-SNE

- Impact of the number of iterations
 - Rule-of-thumb: iterate until you get a stable solution (usually a high number)

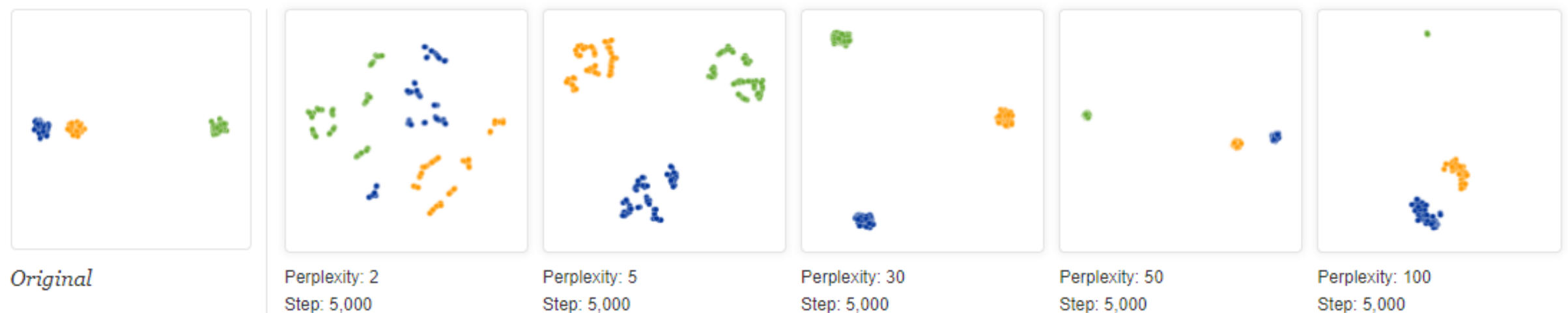


VISUALIZATION WITH T-SNE

- The size of the clusters has no meaning



- The distance between the clusters has no meaning as well



VISUALIZATION WITH T-SNE

- t-SNE plots can be misleading: random data can end up looking like meaningful



VISUALIZATION WITH UMAP

- **UMAP:** Uniform Manifold Approximation and Projection for Dimension Reduction
 - Like t-SNE, UMAP is a nonlinear dimensionality reduction techniques based on manifold learning
 - Parametric technique, so more stable for general data preprocessing compared to t-SNE
 - Proven to work well with high dimensional data like word embeddings and scales better than t-SNE
 - Labels can be given to the mapping process (in essence it created a weighted mapping graph)
 - More info: https://umap-learn.readthedocs.io/en/latest/how_umap_works.html#

VISUALIZATION WITH UMAP

— **UMAP:** Uniform Manifold Approximation and Projection for Dimension Reduction

- First, UMAP constructs a weighted directed graph of the data using a fuzzy distance-based nearest neighbor search (i.e., original graph)
 - The goal is to find local areas where the data points are most likely related
- Next, UMAP translates points in this graph to a lower-dimensional space (i.e., the reduced graph)
 - To make sure that the reduced graph is well approximated, a cross-entropy minimization procedure is performed to minimize the errors between the two graphs
 - As such, the data points are properly matched
- UMAP has better theoretical foundations than t-SNE and only a few parameters need to be tuned
 - The number of nearest neighbors in the original graph
 - The number of dimensions in the reduced graph
 - Base distance metric to use for the minimization procedure (e.g., Euclidean).

TOPIC MODELING

INTUITION

- Topic modeling wants to discover common topics in a corpus
 - Examples
 - What are the main topics in the newspaper?
 - What words are most relevant for a certain movie topic?
 - What words describe a certain Twitter topic and how are they related?
- Most well-known technique: latent Dirichlet allocation (LDA)
 - Latent semantic indexing (LSI) and non-negative matrix factorization (NMF) can also be used

INTUITION

- Suppose you have the following sentences:
 - I like to eat banana and carrots.
 - I ate a banana and spinach smoothie for breakfast.
 - Labradors and cats are cute.
 - My sister adopted a kitten yesterday.
 - Look at this cute dog munching on a piece of carrot.
- LDA = automatically discovering **topics** that these sentences contain
- E.g. 2 topics:
 - Sentences 1 and 2: 100% Topic A
 - Sentences 3 and 4: 100% Topic B
 - Sentence 5: 60% Topic A, 40% Topic B
 - **Topic A:** 30% carrots, 15% bananas, 10% breakfast, 10% munching, ... (at which point, you could interpret topic A to be about *food*)
 - **Topic B:** 20% Labradors, 20% kittens, 20% cute, 15% dog, ... (at which point, you could interpret topic B to be about cute *animals*)

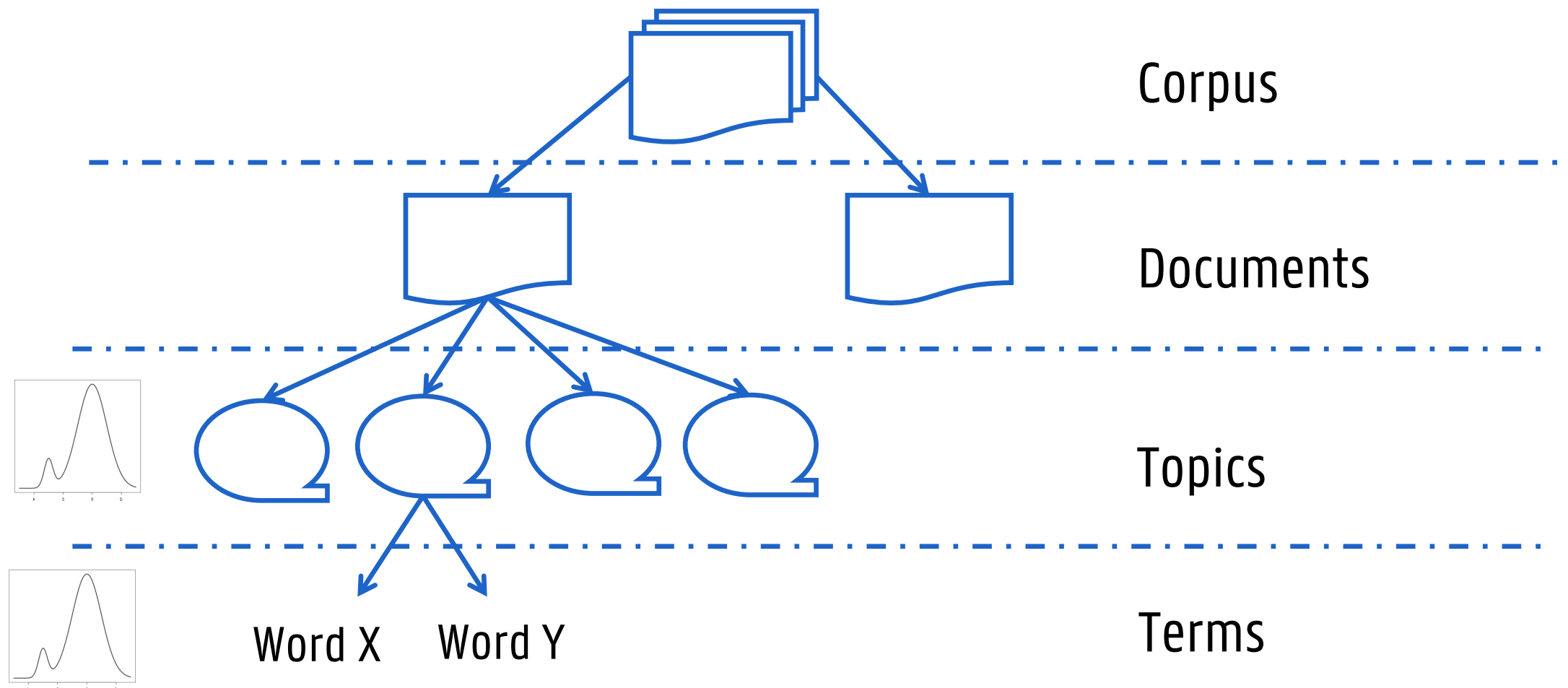
LDA

- The aim of LDA is to find the underlying topics in a document based on the words in that document
- General idea is that documents are generated in two-steps
 1. Randomly generate k topics according to a Dirichlet distribution
 2. For all N words in the document
 - Randomly choose a topic from all topics in step 1
 - Randomly choose a word from its distribution over the vocabulary
- Assumptions
 - Topics are generated before the documents
 - Topics are 'hidden'
 - Does not take into account the order of the words (bag-of-words)
 - All documents have the same topics, but their distribution varies

LDA

— LDA

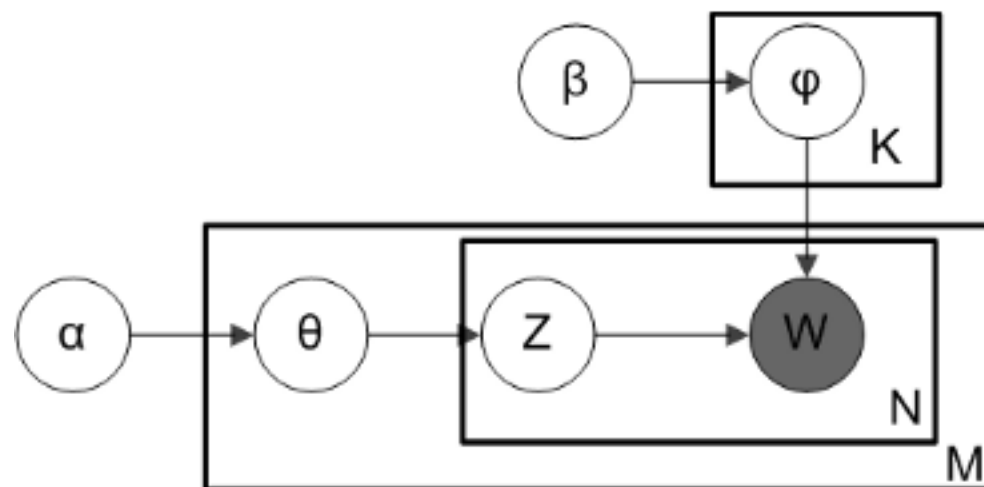
- Documents are represented by a mixture of hidden topics
- One document can contain one or more topics
- A topic: cluster of similar words
- Hierarchical structure



PROCESS

— A generative reversed engineering process

1. Assume there are K topics across all of the documents (α parameter set by the user)
2. Distribute these K topics across document M by assigning a topic to each word
3. For each word W in document M , assume its topic is wrong but every other word is assigned the correct topic.
4. Probabilistically assign word W a topic based on two things ($=p(\text{word } W \text{ with topic } K)$):
 1. $p(\text{topic } K | \text{document } M)$ = the proportion of words in document M that are assigned to topic K , excluding the current word W
 2. $p(\text{word } W | \text{topic } K)$ = how many times word W has been assigned a particular topic across all of the documents (this distribution is called β and is also set by the user)
 3. Update the probability of word W belonging to topic K : $p(\text{word } W \text{ with topic } K) = p(\text{word } W | \text{topic } K) * p(\text{topic } K | \text{document } M)$
5. Repeat this process a number of times for each document and you're done!



PROCESS

- This repetitive process is optimized via Gibbs sampling or variational methods
 - Approximates the posterior probability by searching for topic structures in which the observed documents serve as a guide in the search process
 - In other words, these algorithms will find the optimal α and β
- The α matrix
 - matrix in which the rows are the documents M and the columns the topics K
 - The cells tell you how likely it is that a document M belongs to a certain topic K
- The β matrix
 - A matrix in which the rows are the topics K and the columns the words W
 - The cells tell you how likely it is that topic K contains a certain word W

PROCESS

Topics

gene	0.04
dna	0.02
genetic	0.01
...	

life	0.02
evolve	0.01
organism	0.01
...	

brain	0.04
neuron	0.02
nerve	0.01
...	

data	0.02
number	0.02
computer	0.01
...	

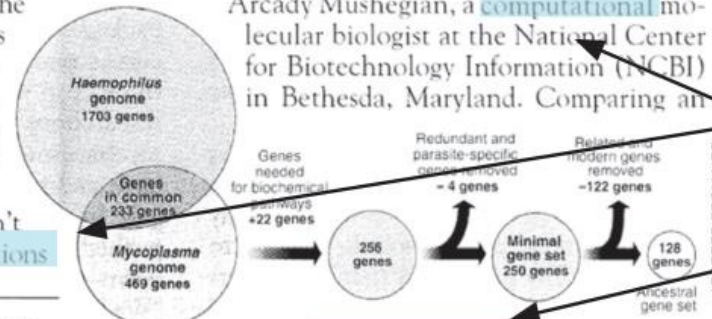
Documents

Seeking Life's Bare (Genetic) Necessities

COLD SPRING HARBOR, NEW YORK—How many **genes** does an **organism** need to **survive**? Last week at the genome meeting here,* two genome researchers with radically different approaches presented complementary views of the basic genes needed for **life**. One research team, using **computer** analyses to compare known **genomes**, concluded that today's **organisms** can be sustained with just 250 genes, and that the earliest life forms required a mere 128 **genes**. The other researcher mapped genes in a simple parasite and estimated that for this organism, 800 genes are plenty to do the job—but that anything short of 100 wouldn't be enough.

Although the numbers don't match precisely, those **predictions**

"are not all that far apart," especially in comparison to the 75,000 **genes** in the human genome, notes Siv Andersson of Uppsala University in Sweden, who arrived at the 800 number. But coming up with a consensus answer may be more than just a **genetic numbers** game, particularly as more and more **genomes** are completely mapped and sequenced. "It may be a way of organizing any newly **sequenced genome**," explains Arcady Mushegian, a **computational** molecular biologist at the National Center for Biotechnology Information (NCBI) in Bethesda, Maryland. Comparing an

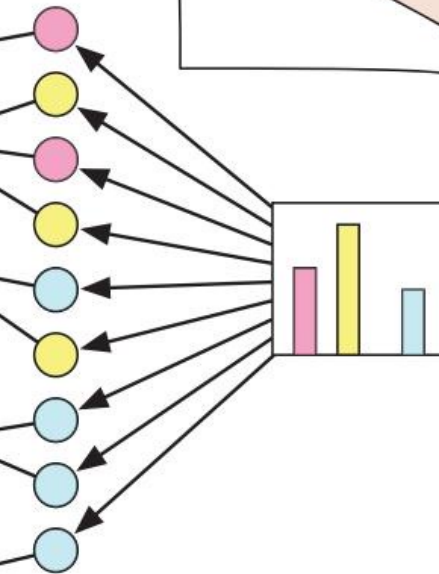


* Genome Mapping and Sequencing, Cold Spring Harbor, New York, May 8 to 12.

Stripping down. **Computer analysis** yields an estimate of the minimum modern and ancient genomes.

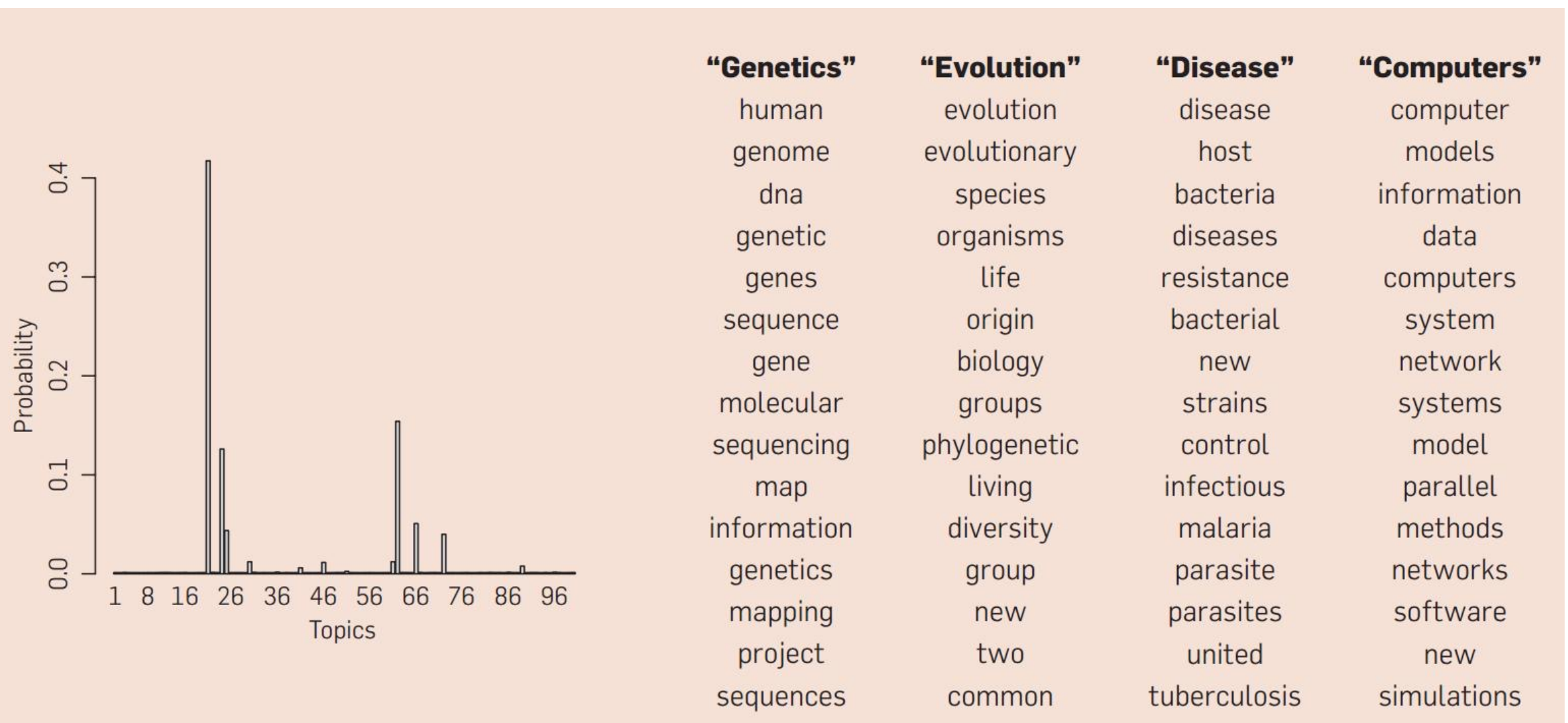
SCIENCE • VOL. 272 • 24 MAY 1996

Topic proportions and assignments



PROCESS

- 100-topic LDA fitted on 17,000 Science articles
- Topic proportions of the example article + 15 most frequent words per topic



DETERMINING THE OPTIMAL K

- A key question in LDA is how many K topics to select
 - Sometimes known a priori, but most often unknown
 - Like with many other unsupervised learning techniques, the ground truth is unknown
- Several options to evaluate the ideal number of K
 - Human Judgements
 - Top N words
 - Topics / Documents
 - Mostly suited for topic interpretation
 - Extrinsic Evaluation Metrics/Evaluation at task
 - Is model good at performing predefined tasks, such as classification?
 - Use one of the well-known performance measures for classification
 - **Intrinsic Evaluation Metrics**

DETERMINING THE OPTIMAL K

— Intrinsic evaluation metrics: coherence

- Coherence scores assess if the words within the topics belong together in a coherent manner
- Normalized point-wise mutual information (NPMI): assumes that topic is more coherent when top words co-occur frequently in corpus

$$- NPMI(word_1, word_2) = \left(\frac{\log \frac{P(word_1, word_2)}{P(word_1) \cdot P(word_2)}}{-\log(P(word_1, word_2))} \right)^\gamma$$

- C_V coherence measure
 - Arithmetic mean of cosine similarities of individual NPMI vectors of top words to general summed up vector
 - Often best representation of human evaluation

DETERMINING THE OPTIMAL K

— Intrinsic evaluation metrics: coherence

— C_{UMass} coherence measure

- Accounts for the ordering among the top N words of a topic
- Assymetric measure: high ranked words ($word_i$) are conditional to higher ranked words ($word_j$)

$$C_{UMass} = \frac{2}{N \cdot (N-1)} \sum_{i=2}^N \sum_{j=1}^{i-1} \log \frac{P(word_i, word_j)}{P(word_j)}$$

- Ready-to-use implementation in R

— Many other coherence measures exist

- Overview: <https://dl.acm.org/doi/pdf/10.1145/2684822.2685324>

DETERMINING THE OPTIMAL K

— Intrinsic evaluation metrics: Likelihood-based

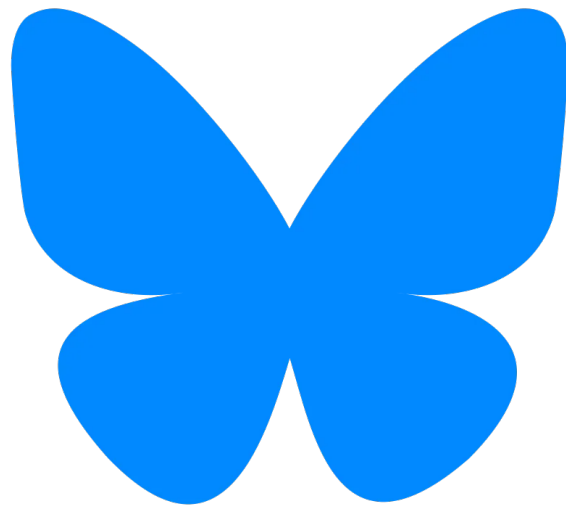
- Likelihood-based scores evaluate plausibility of model given the observed data
- Each measure builds upon the log-likelihood (LL)
- Measures use different ways to penalize the likelihood for model complexity
- Log-likelihood measures how likely the data is to be observed from the assumed distribution (i.e., fitted LDA model): $LL = \log(P(\text{dataset}|\text{model}))$
 - Perplexity: $e^{-LL/D}$
 - AIC: $-2LL + 2p$
 - BIC: $-2LL + p \cdot \log(D)$
 - With $D = \#$ documents and $p = \#$ free parameters in model

REMEMBER

Post about this lecture:

[#SMWA2026](#)

[#Lecture3](#)



SOCIAL MEDIA AND WEB ANALYTICS

Prof Dr Matthias Bogaert